

MPP

PRÁCTICA 1

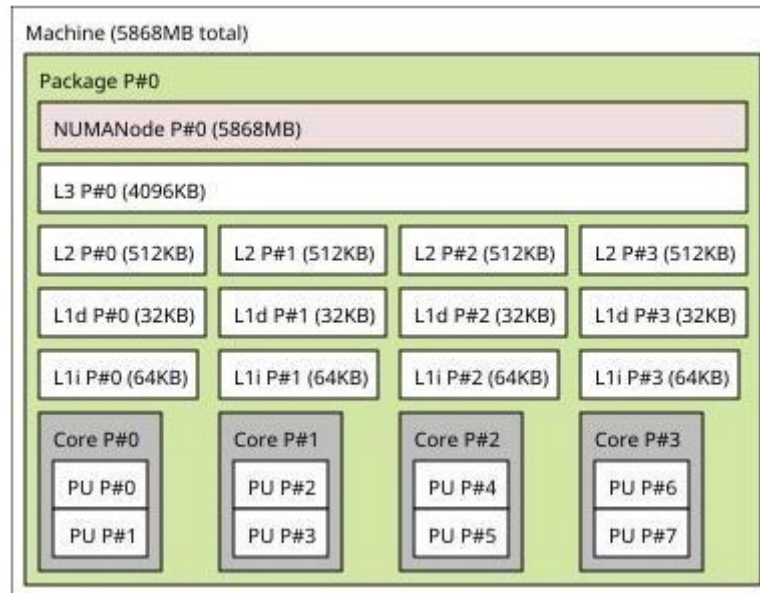
Paralelización de un algoritmo evolutivo usando OpenMP

Curso 2025/26

José Luis García Valverde, subgrupo 1.1

Eric Montalbán Atencia, subgrupo 1.3

Lo primero de todo es mostrar la información del sistema computacional en el que se han realizado las pruebas, para ello y como en la práctica 0, haremos uso del comando: `lstopo -f -p --no-legend --no-io sistema.svg`



En todos los experimentos se ha usado “OMP_NUM_THREADS=8 ./sec 1000 400 200 100 0.15 < ./input_1000_400_200_100.txt”. Lo único variable es el número de hilos.

1ª Semana

Cuestión 1. Justificar qué funciones y/o zonas del código son paralelizables. No es necesario indicar las directivas OpenMP que sería necesario utilizar.

En mh.c:

- **Crear individuo;** no es paralelizable puesto que hay dependencias entre iteraciones, esto se produce puesto que cuando un hilo tiene que acceder a individuo no sabe si el resto han guardado ya sus valores, lo cual genera una dependencia.
- **find_element** Es paralelizable porque se pueden repartir las iteraciones del bucle entre distintos procesos para encontrar si el elemento se encuentra en el array o no. El primer hilo que encuentre el elemento debe comunicarlo al resto para acabar con la búsqueda.
- **Cruzar es paralelizable.** Ambos for se pueden ejecutar de manera paralela puesto que se sabe la posición la posición de corte y en qué posición irá cada gen de antemano. Además, las iteraciones de ambos for también son a su vez paralelizables porque se puede repartir las posiciones del array entre distintos procesos para escribir los genes.
- **Fitness:** es paralelizable, ya que cada cálculo de distancia entre pares de genes (i, j) es independiente del resto. Además de que no existen dependencias de datos ni de control dentro de los bucles anidados, y la única variable compartida (suma) puede gestionarse con una cláusula **reduction** para evitar condiciones de carrera, como se hará en el ejercicio 3.
- **Aplicar_mh:**

```
for (int i = 0; i < tam_pob; i++)
{
    poblacion[i] = (Individuo *)malloc(sizeof(Individuo));
    poblacion[i]->array_int = crear_individuo(n, m);
    fitness(d, poblacion[i], n, m);
}
```

Esta sección es paralelizable puesto que no hay dependencias entre las iteraciones (cada poblacion[i] es distinta y en cada iteracion se crea un individuo

distinto). Por último, no se comparten variables modificables entre hilos, salvo el array poblacion, pero **cada hilo escribe en una posición diferente**.

```
for (int i = 0; i < (tam_pob / 2) - 1; i += 2)
{
    cruzar(poblacion[i], poblacion[i + 1],
    poblacion[tam_pob / 2 + i],
    poblacion[tam_pob / 2 + i + 1], n, m);
}
```

Este bucle es paralelizable puesto que cada llamada a `cruzar()` trabaja sobre **padres e hijos distintos**, sin compartir memoria. No hay comunicación ni dependencias entre las parejas. No se usa ninguna variable global ni acumulador común.

```
for (int i = mutation_start; i < tam_pob; i++)
{
    mutar(poblacion[i], n, m, m_rate);
}
```

Este bucle se puede paralelizar puesto que cada llamada a la función `mutar()` actúa sobre un individuo diferente. No hay dependencias entre iteraciones ni acceso compartido. No se necesita comunicación entre hilos.

```
for (int i = 0; i < tam_pob; i++)
{
    fitness(d, poblacion[i], n, m);
}
```

El cálculo del fitness de cada individuo es **independiente** de los demás. Además, la función `fitness()` solo modifica el campo fitness del individuo correspondiente. Por último, no hay datos compartidos entre iteraciones.

Cuestión 2. Con el fin de obtener un código más eficiente en la parte de memoria compartida y poder optimizar el proceso de mejora de la población de individuos: reemplazar las llamadas a la función `rand` por `rand_r` (que es “thread safe” y se usará para el resto de cuestiones de la práctica) para evitar, por un lado, que se genere la misma secuencia de valores aleatorios en diferentes hilos y, por otro, que se pierda paralelismo durante la generación de dichos valores debido a la secuencialidad que introduce la función `rand`. Además, añadir el código necesario para implementar correctamente la generación aleatoria a lo largo del algoritmo.

Para ello, una posibilidad sería crear en `mh.c` una variable global `seed` que se hará `threadprivate` y que se pasará como argumento por referencia a `rand_r`. Así, además, cada hilo podrá inicializar `seed` en paralelo de forma independiente (por ejemplo, en función del tiempo del sistema, de su identificador de hilo o una combinación de ambos) en una primera región paralela situada justo antes de generar la población inicial de subconjuntos B del problema MDP (bucle `for` que crea cada individuo, que identificaremos como `FOR_INI`). Es en esta generación aleatoria de individuos donde el efecto sobre el paralelismo de `rand_r` en lugar de `rand` es claramente apreciable.

Con el fin de obtener un código más eficiente en la parte de memoria compartida y optimizar el proceso de generación y mejora de la población, se ha reemplazado el uso de la función `rand()` por `rand_r()`, la cual es *thread-safe* y permite generar números aleatorios de forma independiente en cada hilo, evitando la secuencialidad inherente a `rand()`.

De esta manera, se consigue que:

1. Cada hilo genere su propia secuencia aleatoria, evitando duplicaciones entre hilos.
2. No se pierda paralelismo debido a la sincronización interna de `rand()`.
3. Se mejore la escalabilidad y el rendimiento del código al eliminar puntos de serialización.

Para ello, en el fichero `mh.c` se ha introducido una variable global `seed`, que se declara como `threadprivate` mediante la directiva `#pragma omp threadprivate(seed)`, garantizando que cada hilo tenga su propia copia privada y persistente de dicha semilla:

```
static unsigned int seed;  
#pragma omp threadprivate(seed)
```

A continuación, dentro de la función `aplicar_mh()`, se ha añadido una región paralela **antes de la creación de la población inicial (FOR_INI)**.

Esta región inicializa la semilla de cada hilo de forma independiente, cumpliendo con la observación del enunciado que indica que la semilla debe generarse en paralelo utilizando el número máximo de hilos que se empleará durante toda la ejecución:

```
int maxT = omp_get_max_threads(); // es necesario generar la semilla en paralelo con el número de hilos máximo que se vaya a usar en el algoritmo
#pragma omp parallel num_threads(maxT)
{
    // Cada hilo inicializa un seed en paralelo de forma independiente
    unsigned int local_seed = (unsigned int)time(NULL) + omp_get_thread_num(); // TID, id del hilo
    seed = local_seed;
}
```

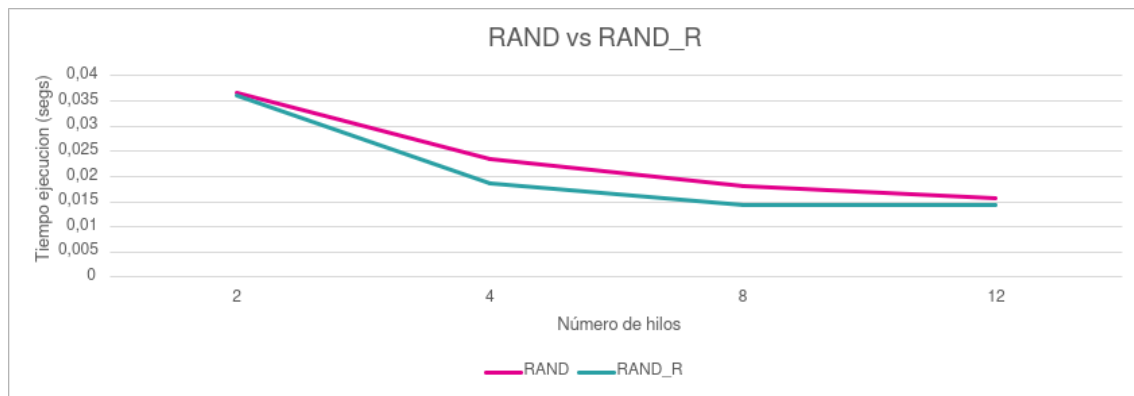
Cada hilo combina el tiempo de ejecución (`time(NULL)`) con su identificador (`omp_get_thread_num()`) para obtener un valor de semilla único.

De esta forma, las regiones paralelas posteriores heredan correctamente su semilla correspondiente, asegurando que los valores aleatorios sean diferentes en cada hilo durante todo el algoritmo.

Finalmente, se ha paralelizado el bucle **FOR_INI**, responsable de la generación de la población inicial, utilizando `#pragma omp parallel for` con una planificación estática:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < tam_pob; i++)
{
    poblacion[i] = (Individuo *)malloc(sizeof(Individuo));
    poblacion[i]->array_int = crear_individuo(n, m);
    fitness(d, poblacion[i], n, m);
}
```

El uso de `schedule(static)` se ha considerado apropiado dado que cada iteración del bucle realiza el mismo tipo de operaciones con un coste computacional similar. Esto permite un reparto equitativo del trabajo entre los hilos.



Como podemos observar en la gráfica hay una diferencia notable entre usar rand y rand_r, siendo la ejecución con rand_r la más rápida a la hora de generar la población inicial. Es con 8 hilos cuando se alcanza el mayor aprovechamiento de los recursos del sistema.

Cuestión 3. Paralelizar la función fitness utilizando, por un lado, los constructores critical y atomic (de forma independiente) y, por otro, la cláusula reduction, indicando en cada caso qué variables serían privadas y cuáles compartidas. Ejecutar el código variando el número de hilos y comparar los resultados obtenidos respecto a versión secuencial utilizando las métricas de rendimiento. Justificar de forma teórica las diferencias y optimizar, en lo posible, el código de critical y atomic teniendo en cuenta cómo está implementada internamente reduction.

Critical

Ahora vamos a realizar la paralelización de la función fitness, para ello primero vamos a usar el constructor critical. A continuación, vamos a mostrar el código de la función con el pragma omp correspondiente:

```

void fitness(const double *d, Individuo *individuo, int n, int m)
{
    double suma = 0.0;

    #pragma omp parallel default(none) shared(d,individuo,n,m,suma)
    {
        #pragma omp for
        for (int i = 0; i < m; i++)
        {
            int elem_i = individuo->array_int[i];
            for (int j = i + 1; j < m; j++)
            {
                int elem_j = individuo->array_int[j];
                double dist = distancia_ij(d, elem_i, elem_j, n);

                #pragma omp critical
                { suma += dist; }
            }
        }

        individuo->fitness = suma;
    }
}

```

Como podemos observar, se ha añadido la directiva `#pragma omp critical`, la cual garantiza que la sección de código protegida sea ejecutada por un único hilo a la vez, asegurando el acceso exclusivo a la variable compartida `suma` durante las operaciones de lectura y escritura.

De este modo, se evita la condición de carrera que podría producirse si varios hilos intentasen actualizar `suma` simultáneamente.

No obstante, el uso de `critical` dentro del bucle interno provoca una gran sobrecarga de sincronización, ya que en cada iteración de dicho bucle se accede a la región crítica. Esto implica que se ejecuta una operación protegida aproximadamente m^2 veces, limitando el paralelismo efectivo.

Para optimizar esta versión, se ha modificado la estructura del código acumulando primero los resultados parciales en una variable local dentro de cada hilo, y desplazando la sección crítica fuera del bucle interno. De esta manera, cada hilo realiza la actualización de la variable compartida `suma` una única vez al finalizar su bloque de trabajo, reduciendo así las operaciones críticas de m^2 a m y mejorando significativamente el rendimiento. A continuación, la versión con `critical` optimizada:


```

void fitness(const double *d, Individuo *individuo, int n, int m)
{
    double suma = 0.0;

    #pragma omp parallel default(none) shared(d,individuo,n,m,suma)
    {
        double dist = 0.0;

        #pragma omp for
        for (int i = 0; i < m; i++)
        {
            int elem_i = individuo->array_int[i];
            for (int j = i + 1; j < m; j++)
            {
                int elem_j = individuo->array_int[j];
                dist += distancia_ij(d, elem_i, elem_j, n);
            }

            #pragma omp critical
            { suma += dist; }
        }

        individuo->fitness = suma;
    }
}

```

Atomic

```

void fitness(const double *d, Individuo *individuo, int n, int m)
{
    double suma = 0.0;

    #pragma omp parallel default(none) shared(d,individuo,n,m,suma)
    {
        #pragma omp for
        for (int i = 0; i < m; i++)
        {
            int elem_i = individuo->array_int[i];
            for (int j = i + 1; j < m; j++)
            {
                int elem_j = individuo->array_int[j];
                double dist = distancia_ij(d, elem_i, elem_j, n);

                #pragma omp atomic
                suma += dist;
            }
        }

        individuo->fitness = suma;
    }
}

```

Ahora pasamos a comentar la implementación usando el constructor atomic, para ello primero comentamos la directiva a usar, `#pragma omp atomic` sobre el código original, que como podemos ver tiene el mismo problema que con `critical`. La solución es la misma, extraer la sección crítica fuera del bucle para reducir la sobresincronización.

```

void fitness(const double *d, Individuo *individuo, int n, int m)
{
    double suma = 0.0;

    #pragma omp parallel default(none) shared(d,individuo,n,m,suma)
    {
        #pragma omp for
        for (int i = 0; i < m; i++)
        {
            int elem_i = individuo->array_int[i];
            for (int j = i + 1; j < m; j++)
            {
                int elem_j = individuo->array_int[j];
                double dist = distancia_ij(d, elem_i, elem_j, n);
            }
            #pragma omp atomic
            suma += dist;
        }

        individuo->fitness = suma;
    }
}

```

Reduction

```

void fitness(const double *d, Individuo *individuo, int n, int m)
{
    double suma = 0.0;

    #pragma omp parallel for reduction(+:suma) default(none) shared(d,individuo,n,m)
    for (int i = 0; i < m; i++)
    {
        int elem_i = individuo->array_int[i];
        for (int j = i + 1; j < m; j++)
        {
            int elem_j = individuo->array_int[j];
            suma += distancia_ij(d, elem_i, elem_j, n);
        }
    }

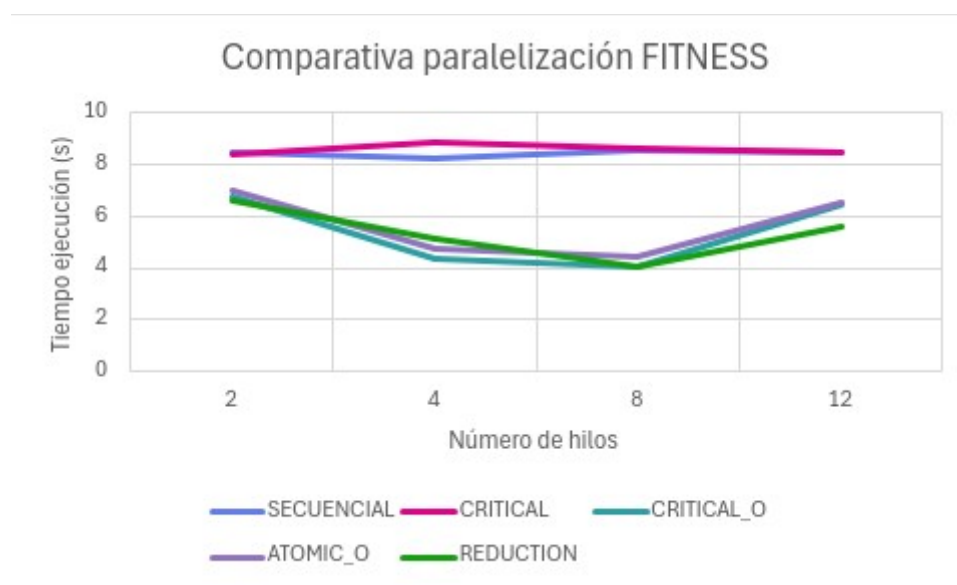
    individuo->fitness = suma;
}

```

En esta versión se utiliza la cláusula `reduction(+:suma)` de OpenMP, que permite realizar la acumulación de la variable `suma` de forma automática, segura y eficiente, evitando el uso de regiones críticas o instrucciones atómicas.

La cláusula reduction crea una copia privada de la variable suma para cada hilo durante la ejecución del bucle paralelo. Cada hilo acumula sus resultados parciales de forma local, sin interferir con los demás, sumando cada par de elementos paralelamente formando un árbol en el que el nodo inferior es el resultado de la suma total. Ésto permite que se calcule de manera óptima la suma. Una vez finalizada la región paralela, OpenMP combina automáticamente todas las copias privadas mediante el operador especificado (+ en este caso), actualizando el valor final de la variable compartida suma.

Gracias a este mecanismo, se evita cualquier condición de carrera, al mismo tiempo que se elimina la sobrecarga de sincronización que introducen las versiones con critical y atomic. Además, esta solución aprovecha al máximo el paralelismo disponible, ya que los hilos no necesitan esperar su turno para actualizar la variable compartida: todas las operaciones se realizan en paralelo y la combinación final se efectúa de forma interna y optimizada por OpenMP.



Como podemos observar en la gráfica, hemos conseguido una mejora significativa al paralelizar la función fitness. Con atomic sin optimizar y dos hilos obteníamos un tiempo de ejecución bastante elevado, en torno a 30s, por lo que hemos optado por no incluirlo en la comparativa. Era peor que la versión secuencial. Sin embargo, al optimizarlo y probando con diferentes hilos se ha reducido el tiempo: entre 4,425s y 7,015s obteniendo mejores resultados que con el fitness secuencial. Por su parte, los tiempos usando critical sin optimizar son similares a los del secuencial, algo más de 8s. Es en su versión optimizada donde se aprecia una diferencia notable de 2s menos de diferencia e incluso 4s respecto a la función fitness secuencial al usar 8 hilos. Finalmente, los tiempos usando reduction son bastante parecidos a los del critical

optimizado, por lo que se obtiene una disminución del tiempo significativa respecto a la versión secuencial.

Por tanto, se han obtenido los mejores resultados con critical optimizado y reduction.

Tabla con los tiempos de ejecución

HILOS	SECUENCIAL	CRITICAL	CRITICAL_O	ATOMIC_O	REDUCTION
2	8,46	8,39	6,775	7,015	6,595
4	8,27	8,86	4,385	4,775	5,11
8	8,595	8,615	4,035	4,425	4,04
12	8,49	8,465	6,48	6,52	5,585

Tabla con los speed-up

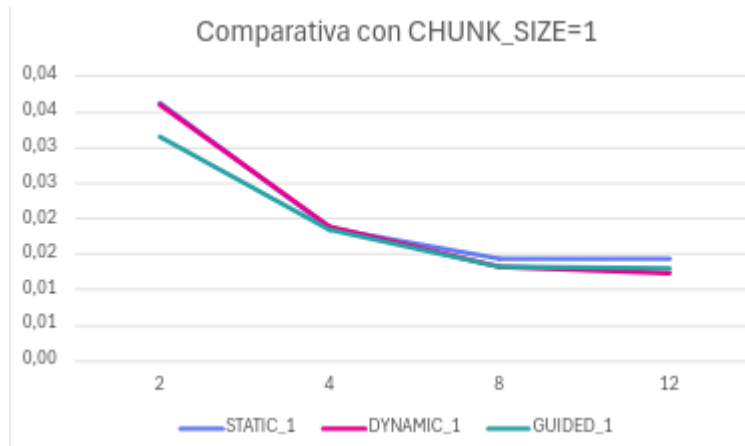
HILOS	CRITICAL	CRITICAL_O	ATOMIC_O	REDUCTION
2	1,008343266	1,248708487	1,20598717	1,282789992
4	0,933408578	1,885974914	1,731937173	1,618395303
8	0,997678468	2,130111524	1,942372881	2,127475248
12	1,002953337	1,310185185	1,302147239	1,520143241

En esta tabla, se muestra un resumen del speedup que se ha conseguido al realizar la paralelización de la función fitness dependiendo del método usado para ello, como podemos ver se confirma que los mejores speedup se obtienen al usar la versión optimizada de critical y la versión con reduction.

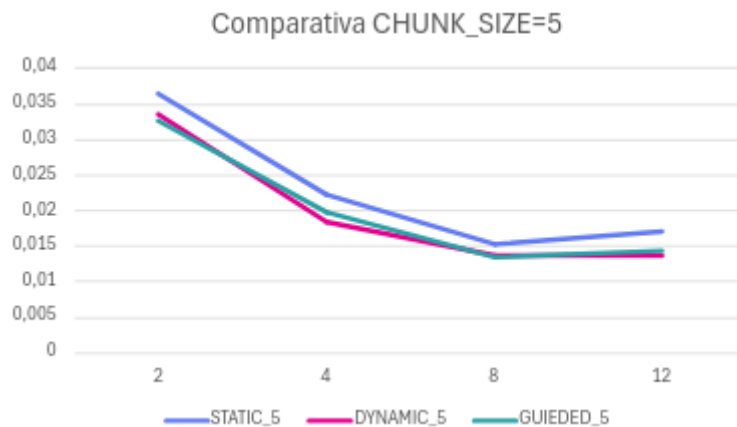
Tabla con la eficiencia

HILOS	CRITICAL	CRITICAL_O	ATOMIC_O	REDUCTION
2	0,504171633	0,624354244	0,602993585	0,641394996
4	0,233352144	0,471493729	0,432984293	0,404598826
8	0,124709808	0,266263941	0,24279661	0,265934406
12	0,083579445	0,109182099	0,10851227	0,126678603

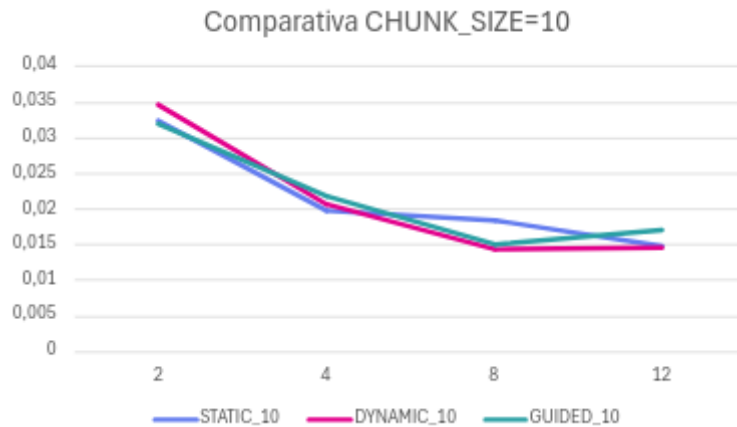
Cuestión 4. Probar diferentes formas de reparto de trabajo, scheduling (static, dynamic, guided), variando el tamaño de asignación (chunk_size) en aquellos bucles en los que se considere necesario. Ejecutar el código variando el número de hilos y justificar los tiempos de ejecución y rendimiento obtenidos con cada política.



HILOS	STATIC_1	DYNAMIC_1	GUIDED_10
2	0,04	0,036054	0,0315195
4	0,0185885	0,018879	0,0183665
8	0,014409	0,0131885	0,0131665
12	0,014349	0,0122635	0,013117



HILOS	STATIC_5	DYNAMIC_5	GUIDED_5
2	0,036428	0,033566	0,0326345
4	0,022337	0,018356	0,019774
8	0,015284	0,0136395	0,0133815
12	0,0170145	0,0136805	0,0142855



HILOS	STATIC_10	DYNAMIC_10	GUIDED_10
2	0,032452	0,0346	0,0320025
4	0,0198685	0,020724	0,0218625
8	0,018337	0,0143225	0,0150535
12	0,014844	0,0146225	0,017068

En OpenMP, la directiva `schedule` controla cómo se reparten las iteraciones de un bucle entre los hilos:

- `static` reparte las iteraciones en bloques fijos desde el inicio. Tiene poco overhead y funciona bien cuando todas las iteraciones tardan lo mismo.
- `dynamic` asigna bloques a los hilos conforme van terminando, equilibrando mejor la carga si las iteraciones son irregulares, aunque con algo más de coste de gestión.
- `guided` comienza asignando bloques grandes y los va reduciendo, buscando un equilibrio entre el reparto dinámico y el overhead.

Al ejecutar el código con distintos números de hilos y tamaños de bloque (*chunk_size*), se observó que el tiempo de ejecución disminuye hasta unos 8 hilos, estabilizándose o empeorando a partir de ahí debido a la sobrecarga de recursos (creación y sincronización de hilos).

En general, las políticas `dynamic` y `guided` ofrecen los mejores tiempos, especialmente con *chunk_size* de 5 o 10, ya que permiten un reparto más equilibrado de trabajo entre los hilos. La política `static`, aunque más ligera, es menos eficiente en este caso debido a pequeñas diferencias en el tiempo de ejecución de cada iteración.

Por tanto, la mejor configuración fue `schedule(dynamic, 5)` con 8 hilos, ya que logra el mejor equilibrio entre carga, overhead y aprovechamiento del paralelismo.

2ª Semana

Cuestión 5. Paralelizar la función cruzar utilizando secciones. ¿Se podría usar la cláusula `nowait`? (argumentar la respuesta). Ejecutar el código variando el número de hilos y justificar el rendimiento obtenido.

Por defecto `sections` incluye una barrera al final de las secciones. El primer bucle `for` de `cruzar` asigna genes a la primera parte del hijo 1 y el hijo 2 mientras que el segundo bucle `for` se encarga de la parte restante de cada hijo. Con la barrera final de `sections`, nos aseguramos de que ambos hijos tengan sus genes correspondientes asignados antes de invocar a `factibilizar` para evitar duplicados. Si usásemos la cláusula `nowait`, podríamos llamar a `factibilizar` antes de que los genes de cualquiera de los dos hijos estén correctamente asignados, puesto que un hilo encargado de un bucle podría terminar antes que el otro, resultando en un cruce incorrecto.

Por último, vamos a mostrar cómo queda la función `cruzar` después de realizar la paralelización usando `sections` sin `nowait`:

```
// CRUZAR PARALELO CON SECTIONS
void cruzar(Individuo *padre1, Individuo *padre2,
            Individuo *hijo1, Individuo *hijo2, int n, int m)
{
    int posicionCorte = (aleatorio(m - 1)) + 1;

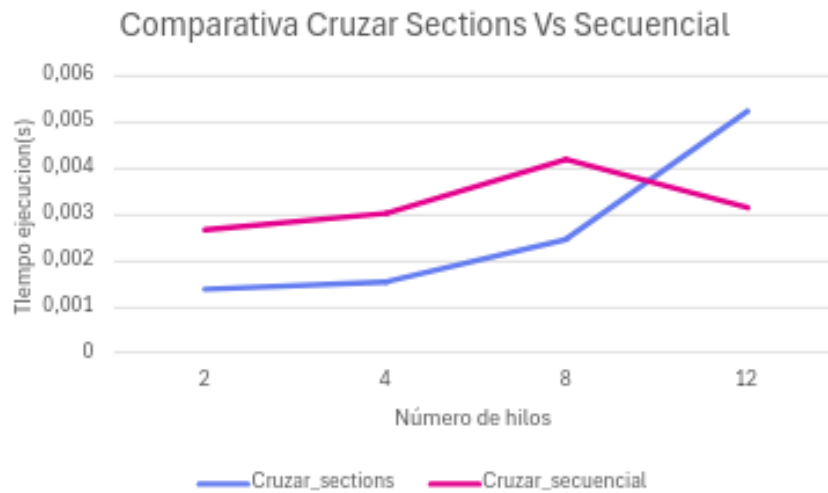
    #pragma omp parallel sections
    {
        #pragma omp section
        for (int i = 0; i < posicionCorte; i++)
        {
            hijo1->array_int[i] = padre1->array_int[i];
            hijo2->array_int[i] = padre2->array_int[i];
        }

        #pragma omp section
        for (int i = posicionCorte; i < m; i++)
        {
            hijo1->array_int[i] = padre2->array_int[i];
            hijo2->array_int[i] = padre1->array_int[i];
        }
    }

    // Barrea implícita al acabar las secciones. Procedemos a factibilizar

    #pragma omp parallel sections
    {
        #pragma omp section
        factibilizar(hijo1, n, m);

        #pragma omp section
        factibilizar(hijo2, n, m);
    }
}
```

HILOS	CRUZAR_PARALELO	CRUZAR_SECUENCIAL
2	0,0013665	0,002668
4	0,001566	0,003048
8	0,0024625	0,004204
12	0,005258	0,003157

Para el experimento hemos medido el tiempo que se tarda en realizar $tam_pob/2 - 1$ cruces en una única generación. Como podemos observar tras la experimentación, la función cruzar paralelizada con secciones es más rápida que la secuencial a excepción del caso con 12 hilos. No obstante, la diferencia entre ambos no es apenas notable, en torno a 1ms.

El uso de sections permite mejorar el rendimiento de la función de cruce cuando se utilizan pocos hilos (2–4), ya que las tareas pueden ejecutarse simultáneamente. Sin embargo, al aumentar el número de hilos, el overhead del paralelismo supera el beneficio, haciendo que la versión secuencial vuelva a ser más rápida. Por tanto, la mejor configuración es la versión paralela con 2 hilos, que logra la mayor reducción de tiempo sin incurrir en sobrecostes de sincronización.

Cuestión 6. Reemplazar las llamadas a la función `qsort` por la función `mergeSort` que se muestra, para ordenar la población por *fitness*, no para ordenar los elementos de cada individuo. A continuación, paralelizar dicha rutina mediante el uso de tareas (OpenMP tasks) y ejecutar el código variando el número de hilos. ¿Se reduce el tiempo de ejecución? Si no es así, explica las posibles causas. La función `mezclar` a la que invoca se proporciona en el fichero `mezclar.c`.

```
void mergeSort(Individuo **poblacion, int izq, int der)
{
    int med = (izq + der)/2;
    if ((der - izq) < 2) { return; }
    mergeSort(poblacion, izq, med);
    mergeSort(poblacion, med, der);
    mezclar(poblacion, izq, med, der);
}
```

Para realizar el cambio en la ordenación de la población, de modo que esta se realice por el valor de *fitness* de cada individuo en lugar de por los elementos internos de cada uno, ha sido necesario integrar en el proyecto el fichero **mezclar.c** proporcionado en los materiales de la asignatura.

Los pasos que se han seguido han sido los siguientes:

1. **Integración de ficheros:** Se añadió el fichero `mezclar.c` a la carpeta `src` del proyecto.
2. **Creación de la interfaz:** Se creó el fichero de cabecera correspondiente, `mezclar.h`, donde se define la interfaz de la función `mezclar()`.
3. **Inclusión de dependencias:** Dado que `mezclar()` hace uso de funciones como `malloc()`, `free()` y `assert()`, además de la estructura `Individuo`, se añadieron los correspondientes `#include <stdlib.h>`, `#include <assert.h>` y `#include "../include/mh.h"` en el fichero `mezclar.c`.
4. **Integración en mh.c:** Se añadió la línea `#include "mezclar.h"` en `mh.c`, con el fin de que las llamadas a `mergeSort()` (que invoca

internamente a `mezclar()` se resolvieran correctamente durante la compilación.

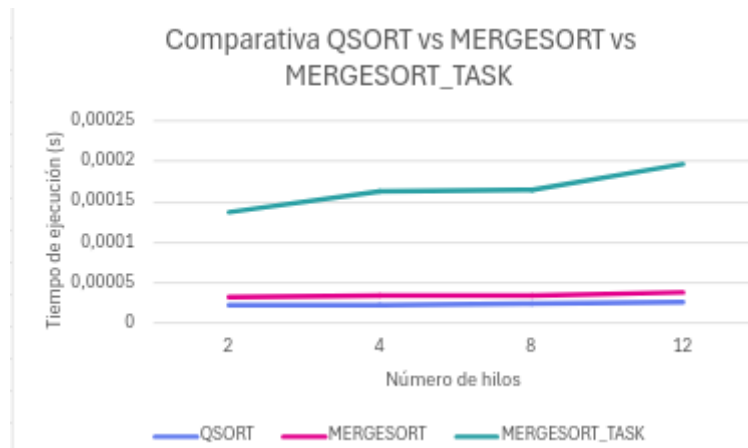
5. **Implementación de `mergeSort`:** En el fichero `mh.c` se incluyó el código de la función `mergeSort()` proporcionado en el enunciado de la práctica, adaptado al tipo de datos `Individuo`.
6. **Modificación del `Makefile`:** Fue necesario modificar el `Makefile` para incluir el nuevo fichero `mezclar.c` durante el proceso de compilación, evitando así errores de enlace.
7. **Sustitución de `qsort`:** Finalmente, se reemplazaron las llamadas a la función estándar `qsort()` por `mergeSort(poblacion, 0, tam_pob)` para que la ordenación se realizara mediante el nuevo método.

Durante la implementación de la función `mezclar()`, se produjo inicialmente un error de violación de segmento (*segmentation fault*). Tras varias pruebas, se comprobó que el problema se debía a accesos a punteros no inicializados dentro del array auxiliar. Para resolverlo, se decidió inicializar explícitamente cada posición del array auxiliar a `NULL` durante su creación, evitando así errores lógicos y accesos indebidos a memoria.

Por último, se procedió a paralelizar la función `mergeSort()` mediante el uso de tareas OpenMP (`#pragma omp task`). Para ello, se añadieron directivas `task` en cada una de las llamadas recursivas al propio `mergeSort()`. Justo antes de la llamada a `mezclar()`, se incluyó la directiva `#pragma omp taskwait`, garantizando que todas las tareas de ordenación recursivas hubieran finalizado antes de realizar la fusión de las subpoblaciones.

Esta paralelización permite que las dos llamadas recursivas a `mergeSort()` se ejecuten de forma paralela en distintos hilos, aprovechando el paralelismo implícito del algoritmo *divide y vencerás* y mejorando la eficiencia en sistemas con múltiples núcleos.

Por último, a pesar de haber paralelizado la función `mergeSort` mediante tareas (`#pragma omp task`), el tiempo de ejecución no se reduce, incluso llega a ser mayor al aumentar el número de hilos (por ejemplo, de 4 a 20). Esto se debe al elevado overhead de creación y sincronización de tareas, junto con el tamaño limitado de la población y la naturaleza recursiva del algoritmo, que genera subproblemas muy pequeños donde el coste de paralelizar no compensa.



HILOS	QSORT	MERGESORT	MERGESORT_TASK
2	0,0000225	0,0000325	0,000138
4	0,0000225	0,0000335	0,000163
8	0,000024	0,0000345	0,000165
12	0,000026	0,0000385	0,000197

El qsort es el más rápido en todas las configuraciones, manteniendo tiempos casi constantes (~ 0.000022 s). Suponemos que se debe a que está altamente optimizado a nivel de biblioteca. El mergeSort secuencial es algo más lento, debido al coste de la recursividad y a que no está tan optimizado como qsort. El mergeSort con tareas presenta los peores tiempos, que aumentan con el número de hilos. Esto ocurre porque el tamaño de los datos a ordenar es pequeño, por lo que el overhead de crear y sincronizar tareas (task y taskwait) supera el posible beneficio del paralelismo.

En este caso, las tareas se crean muy rápido y no hay suficiente carga computacional en cada una como para amortizar la gestión del paralelismo.

Cuestión 7. Explicar para qué se usan en general, y si es necesario utilizar de forma explícita en alguna zona del código los siguientes constructores: barrier, single, master, ordered. Considerar también su posible uso en un código paralelo optimizado (como el requerido en la Cuestión 9).

Barrier no es necesario usarlo en ninguna parte paralelizable del código ya que no es menester que los hilos que queden ociosos esperen al resto para continuar con la ejecución. El único caso donde ocurre ésto es en la paralelización de la función cruzar con secciones, pero como sections ya incluye una barrera implícita no es necesario barrier.

Por su parte, **single** sí se usa en la paralelización de mergeSort ya que se hace uso de tareas debido a la recursividad. Single permite que sólo un hilo sea el que cree las tareas en cada nivel de la recursividad, y que éstas se vayan introduciendo en el pool a medida

que se avanza. Estas tareas se irán asignando a los hilos que estén ociosos. Si no usásemos `single`, se crearían dos tareas por cada hilo que estuviese ocioso en ese momento, aumentando el número de tareas innecesariamente y generando `overload`.

Master permite que la región de código que esté dentro sea ejecutada exclusivamente por el hilo 0. En la región paralela del bucle `for` que itera sobre las generaciones podría usarse para realizar el `qsort`. No obstante, esta opción es más lenta que `single` ya que si el hilo 0 está ocupado hay que esperar a que quede ocioso para pasar a la ordenación. Con `single` el primero que esté ocioso y llegue es el que realiza la ordenación.

Finalmente, la directiva OMP **ordered**, que se usa dentro de una región paralela de tipo `for`. Este constructor es útil cuando queremos que la sección `ordered` se ejecute de la manera en la que se haría con el código secuencial para hacer, por ejemplo, escrituras secuenciales en un archivo o escrituras secuenciales por pantalla. En nuestro caso de estudio del algoritmo genético, no necesitamos hacer nada de lo anteriormente expuesto.

Cuestión 8. Paralelizar la totalidad del algoritmo utilizando las directivas OpenMP que se considere mejor para cada función o bucle (y que pueden ser distintas a las usadas para resolver las cuestiones). Justificar las decisiones tomadas. De este modo, se pretende tener un código paralelo óptimo en memoria compartida que se podrá usar posteriormente para comparar con el código paralelo de paso de mensajes e híbrido implementado en las siguientes prácticas.

A lo largo de esta sección vamos a ver cuáles son las partes que hemos decidido paralelizar y cuales no porque a pesar de que lo sean, debido a que la ganancia sea mínima por lo que pensamos que es mejor dejar el código secuencial al ser más eficiente con el uso de los recursos, puesto que no gastaremos recursos del SO en la creación y sincronización de hilos.

El fichero `io.c` no podemos paralelizar nada, puesto que son lecturas/escrituras secuenciales tanto de un fichero como por la terminal de comandos, esto impide la paralelización.

El siguiente fichero es `main.c`, que no contiene ningún fragmento de código que se puede paralelizar.

A continuación, tenemos el fichero `mh.c`, que es el que se encarga de implementar todas las funciones del AG. En este fichero hay muchas regiones de código que son paralelizables, luego vamos a realizar su posible paralelización y posterior análisis de ejecución para saber hasta qué punto es rentable realizar dicho cambio.

```

int find_element(const int *array, int end, int element) {
    int found = 0;

    #pragma omp parallel shared(found)
    {
        #pragma omp for
        for (int i = 0; i < end; ++i) {

            int f;
            #pragma omp atomic read
            f = found;

            if (f) {
                #pragma omp cancel for
                continue; // salta al punto de cancelación
            }

            if (array[i] == element) {
                #pragma omp atomic write
                found = 1;
                #pragma omp cancel for
            }

            #pragma omp cancellation point for
        }
    }
    return found;
}

```

Se ha paralelizado la función *find_element* utilizando OpenMP. Para ello, se ha introducido una variable compartida llamada *found*, a la que se accede de manera exclusiva con el *atomic read* y *write*. El funcionamiento es el siguiente, cada hilo recorre una parte del array en busca del elemento objetivo y, en caso de encontrarlo, actualiza esta variable de forma atómica. De este modo, el resto de los hilos puede comprobar en cada iteración si *found* ha sido activada y si lo ha sido solicitan la cancelación de la ejecución del bucle *for*, mediante la directiva *#pragma omp cancel for*, que es la manera que tiene OMP de terminar de manera completa todos los hilos que están actuando en la paralelización de un bucle, evitando así realizar trabajo innecesario una vez que otro hilo ya ha localizado el elemento objetivo.

Se han realizado experimentos para valorar la mejora que se obtiene con respecto a la versión secuencial. En este caso el overhead es muy superior a la ganancia por la paralelización, haciendo que el tiempo de ejecución del algoritmo paralelo tarde aproximadamente el doble respecto al secuencial, por lo que optaremos por esta última versión.

crear_individuo no es paralelizable puesto que hay dependencia de datos. El algoritmo de esta función asigna m valores distintos a un individuo. Si se intentara repartir las m iteraciones del bucle entre varios hilos es probable que surgieran valores repetidos.

Las funciones **comp_array_int** y **comp_fitness** no son ideales para paralelización debido a que involucran operaciones atómicas. Esto significa que las operaciones deben ejecutarse de manera indivisible para mantener la coherencia y evitar condiciones de carrera. Como resultado, el coste adicional del manejo de la paralelización, como la creación y sincronización de hilos o tareas, supera con creces cualquier posible ganancia de rendimiento. Por lo tanto, intentar paralelizarlas resulta contraproducente, ya que el overhead generado anula los beneficios.

factibilizar tampoco es paralelizable, al menos de la manera en la que lo hemos implementado secuencialmente. Su paralelización resulta imposible ya que hay dependencia de lectura y escritura secuencial en los genes del hijo.

La función **cruzar** es paralelizable. Hemos pensado dos opciones a la hora de paralelizarla:

```
void cruzar(Individuo *padre1, Individuo *padre2,
            Individuo *hijo1, Individuo *hijo2, int n, int m)
{
    int posicionCorte = (aleatorio(m - 1)) + 1;

    #pragma omp parallel
    {
        #pragma omp for
        for (int i = 0; i < posicionCorte; i++)
        {
            hijo1->array_int[i] = padre1->array_int[i];
            hijo2->array_int[i] = padre2->array_int[i];
        }

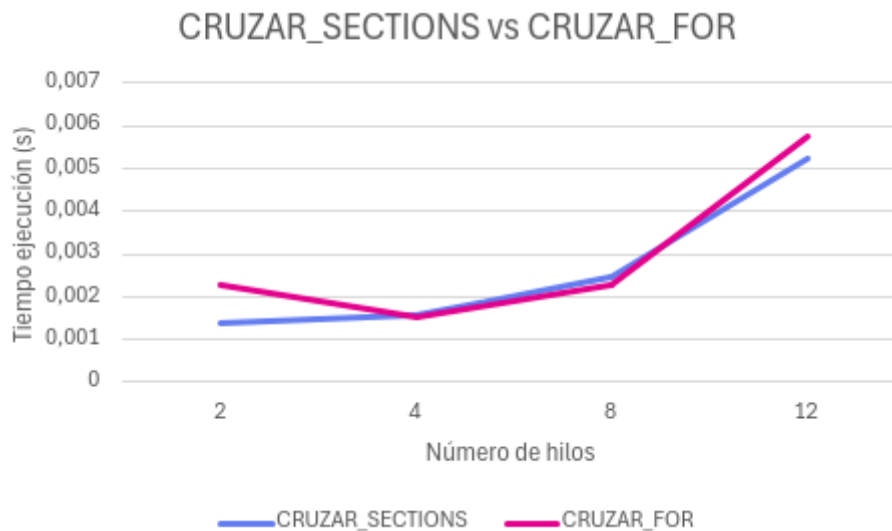
        #pragma omp for
        for (int i = posicionCorte; i < m; i++)
        {
            hijo1->array_int[i] = padre2->array_int[i];
            hijo2->array_int[i] = padre1->array_int[i];
        }
    }

    // Barrea implícita al acabar las secciones. Procedemos a factibilizar

    #pragma omp parallel sections
    {
        #pragma omp section
        factibilizar(hijo1, n, m);

        #pragma omp section
        factibilizar(hijo2, n, m);
    }
}
```

La segunda de las opciones era paralelizar como se indica en el ejercicio 5 con secciones.



HILOS	CRUZAR_PARALELO_SECTIONS	CRUZAR_PARALELO_FOR
2	0,0013665	0,002271
4	0,001566	0,0015455
8	0,0024625	0,002271
12	0,005258	0,0057405

Como podemos ver cruzar con secciones es más estable que paralelizar con for. Sí es cierto que para 2 y 4 hilos con for se comporta mejor, pero la diferencia no es muy notable, por lo que optaremos por la paralelización con secciones.

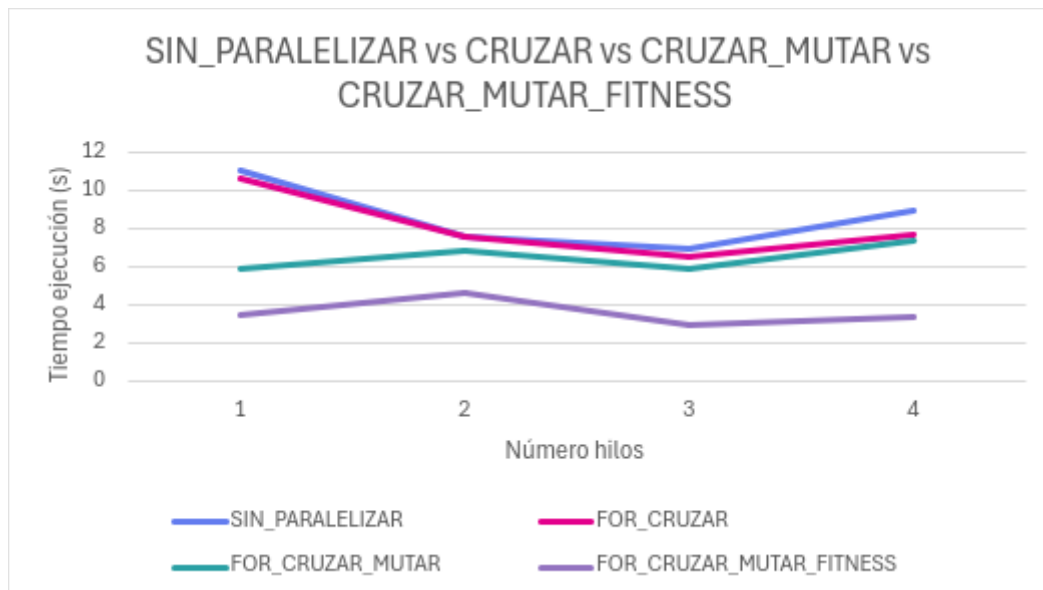
mutar no es paralelizable.

distancia_ij podría ser paralelizable por el cálculo de la fórmula, pero al ser simples operaciones aritméticas produciría overhead.

fitness es paralelizable como bien se indica en el ejercicio 3. Optaremos por la versión con reduction como ya se expuso en el ejercicio anteriormente mencionado.

En **aplicar_mh** se ha realizado la paralelización del bucle FOR_INI usando la directiva #pragma omp parallel for, consiguiendo de esta manera que el tiempo de ejecución del AG se reduzca en 2 segundos.

Dentro de la misma función, en el bucle for que itera sobre el número de generaciones hemos tomado tiempos que resultan al paralelizar los tres bucles for que cruzan, mutan y calculan el fitness obteniendo los siguientes resultados:



HILOS	SIN_PARALELIZAR	FOR_CRUZAR	FOR_CRUZAR_MUTAR	FOR_CRUZAR_MUTAR_FITNESS
2	11,075	10,69	5,875	3,465
4	7,55	7,595	6,845	4,59
8	7,01	6,49	5,875	2,98
12	9,015	7,73	7,33	3,41

Como podemos observar lo óptimo es paralelizar los tres bucles. Con 8 hilos se ha obtenido el mejor resultado: 2.98s, siendo ésta la configuración óptima de todo el algoritmo genético.

A continuación, se muestra cómo quedaría el bucle que itera sobre el número de generaciones:

```

#pragma omp parallel
{
    for (int g = 0; g < n_gen; g++)
    {
        // cruce: reemplaza la segunda mitad
        #pragma omp for
        for (int i = 0; i < (tam_pob / 2) - 1; i += 2)
        {
            cruzar(poblacion[i], poblacion[i + 1],
                poblacion[tam_pob / 2 + i],
                poblacion[tam_pob / 2 + i + 1], n, m);
        }

        // mutación 3/4 de la población
        int mutation_start = tam_pob / 4;
        #pragma omp for
        for (int i = mutation_start; i < tam_pob; i++)
        {
            mutar(poblacion[i], n, m, m_rate);
        }

        #pragma omp for
        for (int i = 0; i < tam_pob; i++)
        {
            fitness(d, poblacion[i], n, m);
        }

        #pragma omp single // master --> más lento porque hay que esperar a que el hilo 0
        {
            //printf("Soy el hilo %d\n", omp_get_thread_num());
            qsort(poblacion, tam_pob, sizeof(Individuo *), comp_fitness); // Ordena la po
            //mergeSort(poblacion, 0, tam_pob);

            if (PRINT)
            {
                printf("Generacion %d - Fitness = %.0lf\n", g+1, poblacion[0]->fitness);
            }
        }
    }
}

```

Cuestión 9. Realizar el informe final incluyendo las conclusiones generales y valoración personal sobre el trabajo realizado.

Esta práctica ayuda al alumno a terminar de comprender la teoría estudiada en OpenMP. Los experimentos tomando tiempos y comparando las versiones secuenciales respecto a las paralelas e incluso la comparación entre distintas versiones paralelas ha sido muy útil para comprender este paradigma de programación. Además, el boletín es una excelente guía para que el alumno se cuestione e implemente el algoritmo genético paso a paso. Para trabajar hemos usado la extensión LiveShare de VScode que nos permite trabajar en paralelo con el código, así como un documento compartido para la realización de este informe. Para los experimentos, uno se encargaba de ejecutar y el otro anotaba los tiempos en una tabla para generar los gráficos en Excel. El tiempo dedicado ha sido de unas 20 horas.